

Extension for scikit-learn: Accelerating workflows with two lines of code

David Cortes, Abel Chaibi



The most widely used Python library
for machine learning on tabular data



4.3 Billion

Total Downloads

97 Million

Downloads/Month

65.8K

GitHub Stars

#1

ML Framework



Scikit-learn (Python)

NumPy / SciPy (Python)

Cython (Python-like C/C++)

BLAS / LAPACK / OpenMP

Hardware (CPU)

- ✓ Readable, testable code
- ✓ Easy to understand
- ✓ Attracts contributors
- ✓ Excellent maintainability



Can we have both?

The Benefit

Readable code

Maintainable

Verifiable correctness

Easy to contribute

The Cost

Hours of waiting

High cloud costs

Limited scalability

Restricted applications



- If scikit-learn does not provide sufficient performance for a use-case, an easy solution is to switch to a different library.
 - For example: 'LightGBM' for gradient boosting, 'glum' for linear models.
- Obvious problem: **degree of compatibility**.
 - Libraries might be scikit-learn compatible, but inputs they accept differ.
 - How can we know that we are building the same model with different libraries?
 - Even if we try to match input parameters, methodologies might differ slightly.
- Other libraries might not test their scikit-learn frontend as well as they test their computational backends.
 - For example: XGBoost offers a scikit-learn interface, but does not recode multi-class 'y' the way scikit-learn does.
 - Might not always work correctly with other scikit-learn-like libraries for feature engineering, hyperparameter tuning, etc.

GradientBoostingRegressor

```
class sklearn.ensemble.GradientBoostingRegressor(*, loss='squared_error',
learning_rate=0.1, n_estimators=100, subsample=1.0,
criterion='friedman_mse', min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_depth=3, min_impurity_decrease=0.0,
init=None, random_state=None, max_features=None, alpha=0.9, verbose=0,
max_leaf_nodes=None, warm_start=False, validation_fraction=0.1,
n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)
```

[\[source\]](#)

Gradient boosting in scikit-learn and in XGBoost.
Are they equivalent? Can we match parameters?

Scikit-Learn API

Does not follow scikit-learn's design

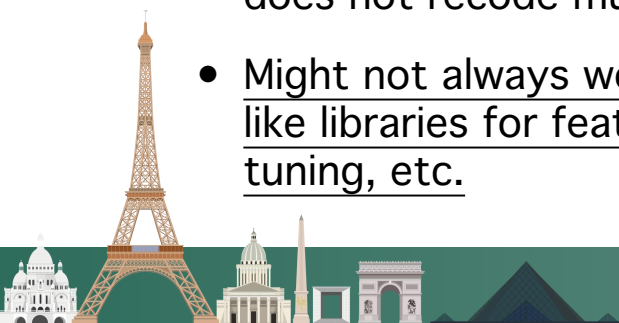
Scikit-Learn Wrapper interface for XGBoost.

```
class xgboost.XGBRegressor(*, objective='reg:squarederror', **kwargs)
```

Bases: `RegressorMixin`, `XGBModel`

Implementation of the scikit-learn API for XGBoost regression. See [Using the Scikit-Learn Estimator Interface](#) for more information.

- Parameters:
- `n_estimators` ([Optional\[int\]](#)) – Number of gradient boosted trees. Equivalent to number of boosting rounds.
 - `max_depth` ([Optional\[int\]](#)) – Maximum tree depth for base learners.
 - `max_leaves` ([Optional\[int\]](#)) – Maximum number of leaves; 0 indicates no limit.
 - `max_bin` ([Optional\[int\]](#)) – If using histogram-based algorithm, maximum number of bins per feature
 - `grow_policy` ([Optional\[str\]](#)) –





BLAS (Basic Linear Algebra Subroutines)

- Standardized interface that every math and HPC lib calls behind the scenes.
- NumPy, PyTorch, scikit-learn all call BLAS routines like 'multiply two matrices' (GEMM).
- Several major implementations: OpenBLAS, Intel's MKL, AMD's BLIS, Apple Accelerate, IBM's ESSL.
- Routines are fine-tuned for specific hardware models (e.g. different block sizes according to CPU cache sizes).

Standard was established decades ago, but ML algorithms in 2026 require different routines.

Example: decision trees have become a lot more popular since the time BLAS was established.

Many LinAlg operations not offered by BLAS/LAPACK have become commonplace in ML (e.g. sandwich products).

What if we could have optimized backends for more complex operations?



- Array API address this gap by allowing to offload common operations to arbitrary array libraries.

For example, PyTorch might provide better performance for many ops than the default NumPy.

Several hardware vendors contribute code branches optimized for their own devices to PyTorch.

Most obvious plus: can also use these arbitrary libraries to offload operations to GPUs.

May execute orders of magnitude faster without code changes.

- Scikit-learn has experimental support for array API.

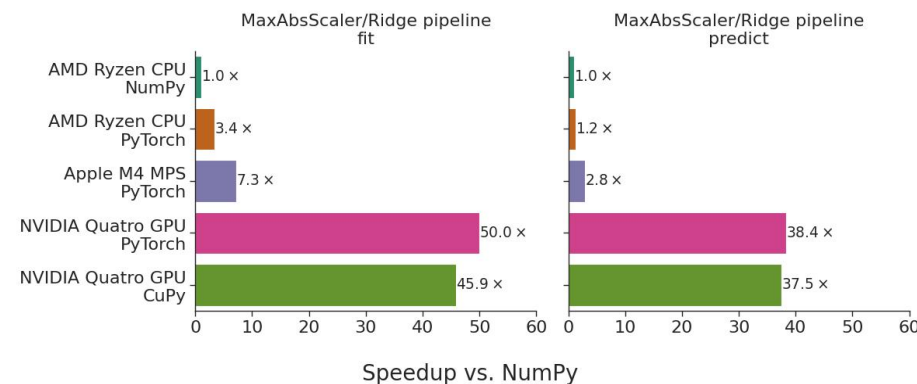
Validated for most common libraries such as PyTorch and CuPy.

Experiments indicate substantial speed ups for some algorithms.

- Problems:

Array API offers a very limited set of operations.

Cannot replace whole algorithms.

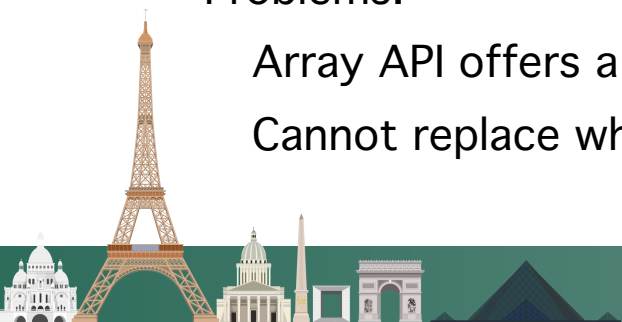


Choose according to your hardware:

PyTorch Build	Stable (2.11.0)		Preview (Nightly)	
Your OS	Linux		Mac	Windows
Package	Pip		LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 12.6	CUDA 12.8	CUDA 13.0	ROCm 7.2 CPU
Run this Command:	pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126			

Source:

<https://blog.scikit-learn.org/updates/update-array-api/>



Could we do better?

- Leaves on each branch will take to random indices of arrays.
 - CPU needs to fetch each index from RAM every time.
 - Repeated random memory accesses become a bottleneck.
 - Better if it could load the indices of the grandchildren together with each child.
 - CPU reads whole cache lines from RAM, not just a single element.
 - Branch predictors/prefetchers make sequential accesses are faster.
 - Idea: rearrange the order of leaves so that children and grandchildren end up together (e.g. Eytzinger layout).
- Do we need an array of indices?
 - If decision tree is full up to N levels, could arrange it so that position of each node is calculable from a math formula (e.g. level N starts at 2^N).
 - One less array to read data from if switched to formula.
- Could reorganize tree structure into chunks/blocks/sequences that follow these ideas.
 - Optimal blocking strategy will vary by hardware.
 - Hardware vendor might know best how to tune to their own products.

Not possible to implement these kinds of operations with array API.

But operation is common enough to warrant optimized implementations.

➤ Latency is critical for domains like online fraud detection.

Scikit-learn's fast prediction route:

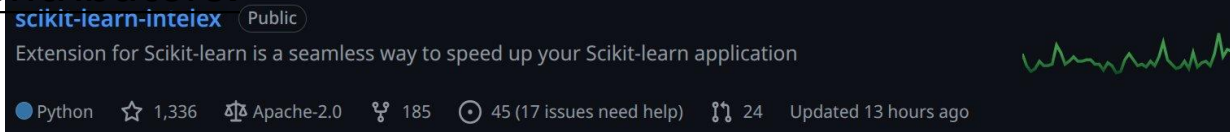
```
cdef void _predict_regression_tree_inplace_fast_dense(
    const float32_t[:, ::1] X,
    Node* root_node,
    double *value,
    double scale,
    Py_ssize_t k,
    float64_t[:, :] out
) noexcept nogil:
    """Predicts output for regression tree and stores it in `out[i, k]`.
```

Same model but served under different frameworks

	Prediction time (P99) for single row (ms)			
Dataset	xgboost	treelite	onnx	scikit-learn-intelex
a9a	370	95	20	19
airline	406	97	18	16
airline-ohe	1653	255	76	69
california_housing	236	68	14	14
creditcard	244	93	17	16
higgs	2215	268	139	62
msrank	268	94	17	17
year_prediction_ms	244	91	12	12
Speedup vs xgboost		4.0	18.0	20.8



- **Extension for scikit-learn** introduces highly optimized versions of selected procedures that can be seamlessly integrated into existing scikit-learn workflows:
 - <https://github.com/uxlfoundation/scikit-learn-intelex>
- Idea: accelerate *only* calls to selected compute-heavy functions and methods inside scikit-learn:
 - System of backends for internal methods handles integration:
 - No need to rewrite whole classes, reuse scikit-learn's where appropriate.
 - More flexible than array API, applicable to much broader set of algorithms.
- Library was initially developed by Intel as 'daal4py', then renamed to 'Intel Extension for scikit-learn', but has since been transferred to the UXL foundation (<https://uxlfoundation.org>).
 - Support for ARM and RISC-V has been contributed by other UXL members since initial Intel development.
 - Library is fully open source and open for external contributors



```
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
X, y = make_regression(
    n_samples=1_000_000,
    n_features=15,
    random_state=123,
)
model = LinearRegression().fit(X, y)
model.coef_
✓ 1.2s

array([ 3.51026213e+01,  3.70474128e+01, -1.99840144e-14,  7.73831526e+01,
        4.03390845e+01,  6.88734308e+01,  3.28626015e-14,  2.93449866e+01,
       -2.22044460e-16,  3.15443306e+01,  3.11785094e+01, -2.44249065e-14,
        7.00969187e+01, -9.01501096e-14,  2.78642875e+01])

from sklearnex.linear_model import LinearRegression as AcceleratedLinearRegression
model2 = AcceleratedLinearRegression().fit(X, y)
model2.coef_
✓ 0.1s

array([ 3.51026213e+01,  3.70474128e+01,  7.00386161e-17,  7.73831526e+01,
        4.03390845e+01,  6.88734308e+01,  1.19613575e-15,  2.93449866e+01,
        3.73636392e-16,  3.15443306e+01,  3.11785094e+01, -1.36419214e-15,
        7.00969187e+01, -2.76304433e-15,  2.78642875e+01])
```

Just a one-line change:
import from sklearnex instead of sklearn.

Results match up to roundoff error.

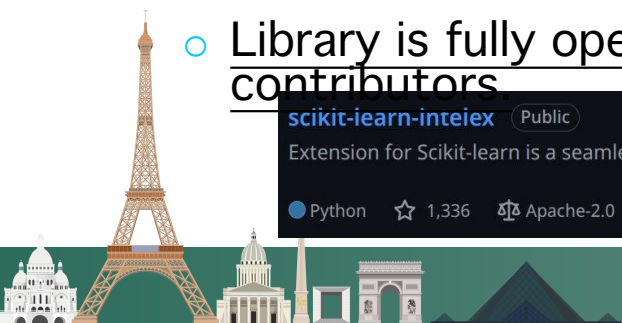
But one is faster than the other:

```
%timeit
LinearRegression().fit(X, y)
✓ 2.7s

333 ms ± 15.3 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

%timeit
AcceleratedLinearRegression().fit(X, y)
✓ 18.1s

23.4 ms ± 4.51 ms per loop (mean ± std. dev. of 7 runs, 100 loops each)
```





Unified Acceleration Foundation - Multi-architecture,
multi-vendor open standards

intel®

SAMSUNG

arm

Qualcomm

Google

FUJITSU

BROADCOM®

Imagination

GE HealthCare

Key Projects:

oneMath
BLAS/LAPACK

oneDNN
Deep Learning

oneDAL
ML Algorithms ← Powers
scikit-learn-intelx!

oneTBB
Threading



- Estimators are subclassed from scikit-learn.
- Methods like '.fit()' are overridden with a dynamic dispatching mechanism.
 - Dispatcher forwards users calls to different branches (e.g. scikit-learn's original '.fit' vs. accelerated method) depending on conditions.
 - Checks are performed on data and parameters to figure out if accelerated backend supports what's requested.
 - Logging system records the outcomes from these checks for more information.
 - Checks on scikit-learn version encountered at runtime allow different branches to match exact behavior of a library in continuous development.
 - Not bug-for-bug compatible, but useful for changes like parameter renamings.
- **Not just scikit-learn-*like*, code is mostly scikit-learn itself.**
 - Validated against scikit-learn's own test suite.
 - Estimators have the same public-facing methods and attributes as scikit-learn's.
 - Fallback mechanism ensures every functionality that scikit-learn offers is available, just not always accelerated.

```
def fit(self, X, y, sample_weight=None):
    if sklearn_check_version("1.2"):
        self._validate_params()

    dispatch(
        self,
        "fit",
        {
            "onedal": self.__class__.__onedal_fit,
            "sklearn": _sklearn_LinearRegression.fit,
        },
        X,
        y,
        sample_weight,
    )
    return self
```

```
is_multi_output = _num_features(y, fallback_id=True) > 1

patching_status.and_conditions(
    [
        (sample_weight is None, "Sample weight is not supported."),
        (
            not is_sparse(X) and not is_sparse(y),
            "Sparse input is not supported.",
        ),
    ],
```

```
mapping = {
    "sklearn.cluster.DBSCAN": (
        cluster_module,
        "DBSCAN",
        DBSCAN_sklearnex,
        DBSCAN_sklearn,
    ),
    "sklearn.cluster.KMeans": (
        cluster_module,
        "KMeans",
        KMeans_sklearnex,
        KMeans_sklearn,
    ),
```



Installation

```
pip install scikit-learn-intelex
```

Option 1: Change imports

```
from sklearnx.ensemble import \
    RandomForestRegressor
model = RandomForestRegressor()
model.fit(X, y)
```

Option 2: Monkeypatch

```
from sklearnx import patch_sklearn
patch_sklearn()

from sklearn.ensemble import \
    RandomForestRegressor # Accelerated!
```

Additional Features

✓ GPU Support

✓ Distributed Mode (MPI)

✓ XGBoost/LightGBM
Serving

✓ Extra Estimators

Resources

uxlfoundation.github.io/scikit-learn-intelex

github.com/uxlfoundation/scikit-learn-intelex

uxlfoundation.org

